Chapter 27 worked solutions

Time-Series Forecasting

Exercises 1–4 work through what a forecast can use, how to build its windows, and how to score its intervals. Exercises 9–11 apply those ideas to evaluation design.

1. Separate what is known from what will arrive

At Thursday's close, the shop can look up Friday's schedule even though Friday has yet to arrive. A covariate is an input that helps predict the quantity of interest. Its availability depends on when the shop gets the value.

Value	Type of input	Available at Thursday's close?
Store size	Static covariate, a store characteristic that stays fixed over the forecast window	Yes
Friday's scheduled opening hours	Known future covariate, since the schedule is already published	Yes
Thursday's sales	Past observation, assuming the sales total is recorded at close	Yes
Friday's observed foot traffic	Future observation, measured on Friday	No

2. Keep targets inside their split

The last training window must end at or before index 15. Its three target indices are 13, 14, and 15, so the **last training origin is 13**. The origin is the index of the first target. Moving it forward to 14 would produce targets at 14, 15, and 16, crossing the split.

The **first test origin is 16**, with targets at 16, 17, and 18. At that point the four observations at 12 through 15 are already available, so the model can use them as its lookback. The split is determined by the target indices.

Window	Origin	Past indices	Target indices
Last training window	13	9, 10, 11, 12	13, 14, 15
First test window	16	12, 13, 14, 15	16, 17, 18

3. Trace overlapping patches

Start at index 0, take three consecutive values, and move the start two indices at a time.

Patch	Indices	Values
First	0, 1, 2	4, 2, 6
Second	2, 3, 4	6, 1, 7
Third	4, 5, 6	7, 3, 8

The first and second patches share index 2, whose value is 6. The second and third share index 4, whose value is 7. A fourth patch starting at index 6 would need index 8, which is outside the series. Index 7, whose value is 5, therefore appears in no complete patch.

Each patch supplies one token. With dense attention, each of the three query positions pairs with all three key positions, giving **$3 \times 3 = 9$ ordered pairs**. Three are self-pairs. The other six include both directions for each pair of distinct patches.

4. Count interval hits

The actual value 6 is on the upper endpoint of the first interval, $[2, 6]$, so it counts as a hit. The value 8 is above the upper endpoint of $[4, 7]$, so it is a miss. Empirical coverage is the fraction of actual values inside their own forecast intervals. Here that is **1 out of 2, or 50%**.

Each width is its upper endpoint minus its lower endpoint. The first width is $6 - 2 = 4$ and the second is $7 - 4 = 3$. The **mean width is** $(4 + 3)/2 = 3.5$ **units**.

9. Audit feature availability

Suppose the forecast is issued just before day o, the first unknown day, and it predicts demand through day $o + 2$. Sales through day $o - 1$ have been recorded. A feature dated $o - 1$ might look safe because its timestamp is in the past, while its calculation uses sales we will only learn later.

A centered seven-day sales average at day $o - 1$ includes three days on each side. It uses days $o - 4$ through $o + 2$. The last three entries are exactly the forecast targets. Even if the dataset stores the average in a historical row, it leaks those targets into the input. Use a trailing seven-day average instead. At origin o, compute it from days $o - 7$ through $o - 1$. If sales are finalized after an extra day, end the window at the latest value that was actually available. Define what happens at the start of the series if fewer than seven observations have arrived. The forecast can reuse this summary across horizons. Computing a new trailing average for each future day would eventually require sales that have not happened yet.

The weather column has a different problem. A weather forecast issued before o for day $o + 2$ could enter the model. Actual weather measured on $o + 2$ could not. Overwriting the forecast with the measurement destroys the distinction. The same column can then appear more accurate during training and retrospective testing than it could be in production.

Keep each weather forecast's issue time, arrival time, location, and target date, and store measured weather in a separate field. For each origin and future date, use the latest forecast for that date that had arrived by the issue time of the sales forecast. For example, an update issued during day $o + 1$ is too late for the original forecast, even if it is the latest record for day $o + 2$. If a historical forecast version is missing, apply a predefined missing-value rule using available information. Reconstruct features with these same time checks for training, validation, and test. Inspect a few origin records and verify the timestamps of the raw observations and forecast versions they read.

10. Choose a rolling protocol

Consider a shop that makes a forecast for the next three days each morning. An origin is the index of the first target day. Start with a single daily series and split it in time order.

Period	Day indices	Use
Training	0–167	Fit parameters and data transformations.
Validation	168–195	Choose lookback, model settings, and the weekly refit procedure.
Test	196–251	Compare the two frozen procedures at the same origins.

A window belongs to a split only if all three of its target days are inside it. During validation, replay the two candidate procedures in order, using only information that had arrived at each origin. Choose settings on validation and freeze them before test. Just before day 196, combine the training and validation periods into one fitting pool. Fit both systems using the same chosen settings and all complete rows with targets inside that pool and observed by then.

Generate one forecast at each test origin from 196 through 249, for a total of 54 forecasts per system. At 249, the targets are 249, 250, and 251. Origins 250 and 251 would need targets beyond the test period. Give both systems the same available history and covariates, such as past sales and foot traffic, future day of week, and the promotion schedule known that morning. Future measured foot traffic is withheld. Preserve as-issued versions of any changing plan or forecast. As soon as the previous day's actual sales have arrived, both systems may read them as part of the next input history.

The **fixed** system retains the parameters and fitted transformations from day 196. Updating its history is part of forecasting with those parameters. The **weekly refit** system updates at origins 203, 210, and every seven days through 245, for seven refits. At each refit it recomputes the fitted data transformations and model parameters with an expanding training set through the previous day. Only rows whose targets have all arrived enter a refit. Between weekly updates it keeps the latest fitted parameters. Observations from earlier in test are therefore allowed for later refits under this protocol. Neither system can revise a forecast already issued or use results from later origins to choose its settings.

Report MAE separately at each horizon on the same test origins, and compare the errors pair by pair. Also report initial fit time, number of refits, time per refit, total update time, and per-origin forecast latency. Time on the same hardware with the same batch and threading settings. State whether the weekly fit has to finish before that day's forecast can be published. This distinguishes ordinary inference latency from the additional compute and delay due to updating the model.

11. Evaluate intervals under a shift

Let day d be the first day when the new depot serves deliveries. A forecast issued before d can include both old- and new-regime targets. For example, a seven-day forecast issued at $d-2$ includes two days before opening and five days after it. Choose how to label these cases before scoring them.

For the primary comparison, use the same forecast origins for both horizons. A **before** origin has all seven target days strictly before d. An **after** origin has all seven on or after d. This rule uses the target window. At the first after origin, the forecast may legitimately read history from before the depot opened. Keep origins whose target windows straddle d in a separate transition group and report them separately. For example, horizon one at $d-2$ targets day $d-2$, but horizon seven targets day $d+4$. Treating that row as entirely before or entirely after would mix conditions across the horizons. Fix the length of the study periods and consider weekday balance before inspecting the results.

Fit the point forecaster before the study. Use forecasts it made on a separate pre-opening calibration period to set interval rules. Wait for all those calibration outcomes before the first evaluation origin. For example, estimate the 10th and 90th percentiles of the signed residual $y-\widehat{y}$ separately for horizons one and seven. Add those offsets to the corresponding point forecast to produce lower and upper endpoints. The offsets set a nominal 80 percent interval, whose observed coverage must be checked. Freeze the offsets before generating and scoring either test period. At each test origin, save the interval and its issue time before the outcome arrives. Keep the point model's fitting and history policy the same across the comparison. Actuals from the post-opening evaluation do not enter the calibration.

For each horizon and period, count how many actuals fall inside their interval, including its endpoints, and divide by the number of forecasts. Also average the upper endpoint minus the lower endpoint. For instance, 16 hits out of 20 give 80 percent coverage, and 12 out of 20 give 60 percent. Report the counts with the percentages. With fixed additive offsets, width is constant within a horizon, so a drop in coverage shows more misses at the same width. Report the before, after, and transition results side by side, noting how many origins were available for each. These scores describe calibration across the opening under the chosen protocol.